

Improving Automatic Verification of NL2SQL Queries: an Avenue for Exploration

Junho Won¹, Donghyun Sohn¹

¹Northwestern University



Northwestern
University

Background & Related Work

How do we generate more accurate NL2SQL (Text-to-SQL) queries with LLMs?

- Generation-side approaches focus on improving the generator itself, through schema linking (CHESS, MAC-SQL, CodeS), query decomposition (DIN-SQL, MAC-SQL), or fine-tuning on synthetic SQL (CodeS, SENSE).
- Selection-side approaches generate N candidates and rerank with a critic or learned scorer (Alpha-SQL, MCTS-SQL, RSL-SQL), leveraging candidate diversity to push accuracy beyond what single-shot achieves.
- Self-correction methods refine SQL across iterations using execution feedback or LLM critique.
- **Verifiers themselves remain mostly zero-shot LLM critiques with prompt-based decision rules.** Their calibration and failure modes have received limited systematic study.

Motivation: NL2SQL Verification

- Reranking achieves strong accuracy on BIRD, but generating N candidates in parallel is expensive at inference time.
- Self-correction with a binary verifier offers a cheaper alternative. The generator produces one SQL, the verifier judges it, and only failures trigger regeneration. Effectiveness depends entirely on verifier reliability
- **In real-world databases, ground truth is unknown and highly accurate verifiers are needed for scaling NL2SQL workflow.**

Approach: SQL Verifier SFT

- We applied supervised fine-tuning (SFT) to gemma-4-E4B-it to act as a binary SQL correctness verifier.
- Given a natural-language question, schema context, a candidate SQL query, and database-only execution feedback, the model predicts whether the SQL is correct.
- LoRA (rank 16, dropout 0.05) on HuggingFace TRL

SFT Training Data Generation

- Gemini-3-flash-preview was used to generate 2858 rows of candidate SQL queries and reasoning based on 250 questions in the BIRD-bench mini-dev dataset.

Split	Questions	Rows	Correct	Incorrect	Correct %
SFT train	250	2858	861	1997	30.1%
Test	50	537	133	404	24.8%

Train and test data

```
Question:
What is the total number of superheroes without full
name?
Hint:
superheroes without full name refers to full_name IS
NULL
Candidate SQL:
SELECT COUNT(*) FROM superhero WHERE
superhero.full_name IS NULL;
Execution:
exec_ok=True
row_count=1
sample_rows=[ '(122,)' ]
Target:
Verdict: correct
Type: none
Reason: SQL executes correctly and matches expected output.
.....
```

Sample of training data

- Input: verifier prompt text containing question, hint, schema, candidate SQL, SQL structure, execution result, mismatch signals
- Truth label: binary label, true = SQL correct, false = SQL incorrect

Prompt Bias Ablation

We tested whether verifier behavior was driven by the fine-tuned model itself or by a conservative system prompt.

Non-neutral / conservative prompt:

Use `predicted\_correct=1` only when every explicit requirement is satisfied.

Execution success is weak evidence only.

When uncertain, prefer `predicted\_correct=0`.

Neutral prompt:

Set `predicted\_correct=1` when the SQL semantically answers the question;

set `predicted\_correct=0` when it does not.

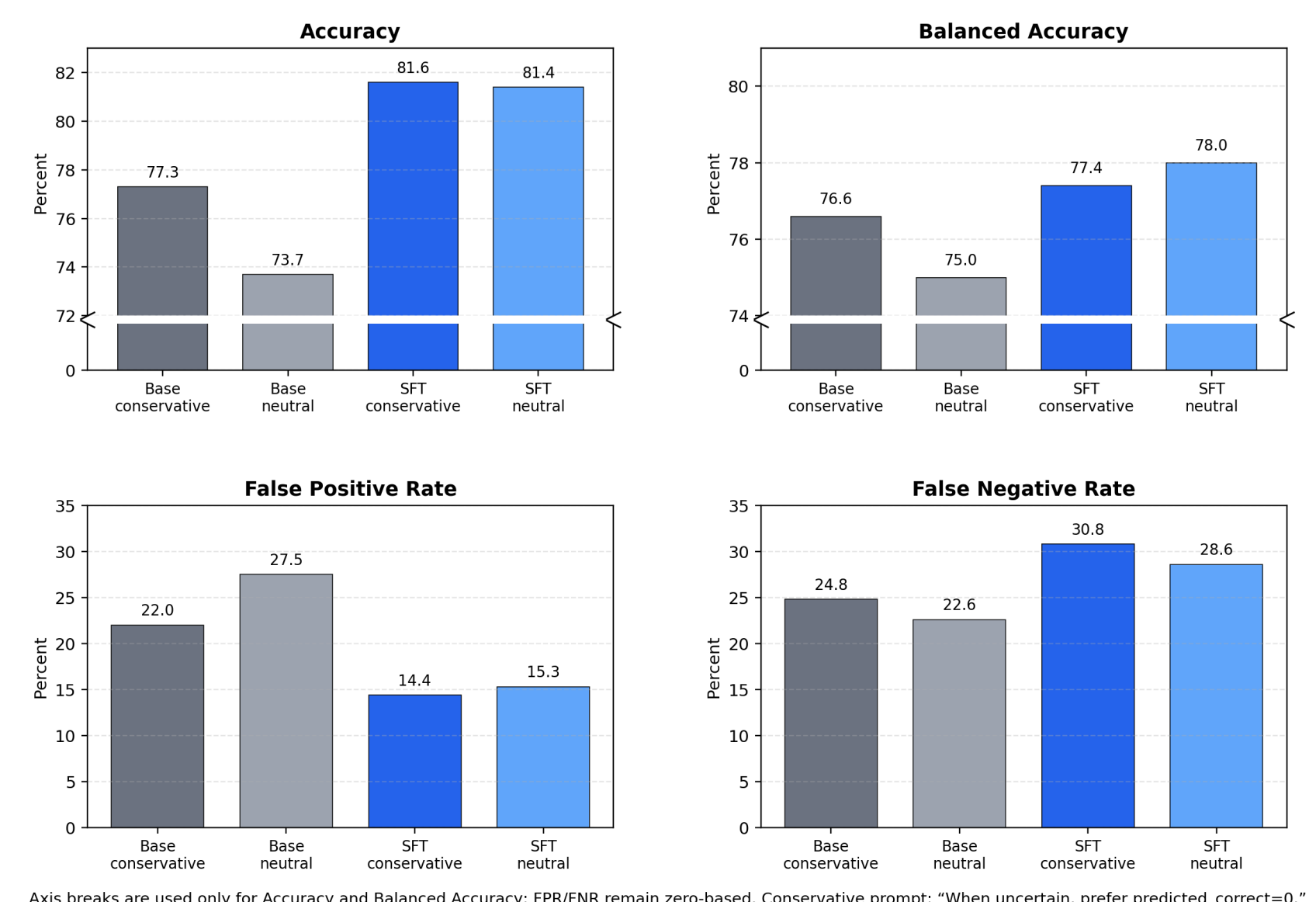
System prompts for ablation study

- Result: the conservative prompt encouraged stricter verification and reduced false positives, while the neutral prompt removed this explicit bias but could shift the model toward different accept/reject tradeoffs after SFT. This ablation tests whether rejection behavior comes from learned evidence use or from prompt-level conservatism.

Results

Model	Acc.	Bal. Acc.	FPR	FNR
Base, conservative	77.3	76.6	22.0	24.8
Base, neutral	73.7	75.0	27.5	22.6
SFT, conservative	81.6	77.4	14.4	30.8
SFT, neutral	81.4	78.0	15.3	28.6

Held-Out Test Performance (n=537)



- SFT improves held-out accuracy mainly by reducing false positives: FPR drops from 22.0% to 14.4% with the conservative prompt, and from 27.5% to 15.3% with the neutral prompt.
- After SFT, conservative and neutral prompts are very close in accuracy. The conservative prompt gives the lowest false-positive rate, while the neutral prompt gives slightly better balanced accuracy and lower false-negative rate. This suggests most of the improvement comes from fine-tuning, not only prompt-level conservatism.

Future Work

- This is an on-going and preliminary work! We need to test on more diverse data and build a generalizable verifier model.
- Further develop and compare different approaches for synthetic training data generation.
- Study effects and training dynamics of different Reinforcement Learning algorithms (GRPO, RLVR) in NL2SQL Verifier training
- Adapt ideas used in NL2SQL generation research for verification: e.g. using DB profiling or query-log analysis (AskData), or in orthogonal direction, test-time scaling of SQL generation and selection (Agentar-Scale-SQL).